

Runbook de despliegue — Fase 1 Arrendadores

Destino: 209.97.146.136 · `/var/www/Spaces` · proceso PM2 `spaces-web` Rama: `main` — asume que `feat/arrendadores-fase1-prod` (punta `36acb4a`) ya está mergeada, como se hizo con el PR #8 Fecha: 2026-07-31 (revisión 2; la 1 fue del 2026-07-30)

Qué se despliega

Lo de la revisión 1:

- **ADR 0006 Fase 1** — un solo costo por pantalla: la renta al arrendador.

`costo_compra` deja de capturarse y pasa a ser espejo de la renta.

- **Firma electrónica del contrato** — el documento se congela y se sella con

SHA-256; enlace público por token para el arrendador.

- **Razones sociales del arrendador** — alta, edición y borrado.
- **Renta** — captura masiva, periodicidad (incluida DIARIA) y recordatorios

proporcionales a la cadencia.

Y los seis arreglos de la revisión del flujo (2026-07-31):

- **Las fechas de calendario se pintaban un día antes.** `pagos_renta.periodo` es

`text` y `formatFecha` lo leía como medianoche UTC. Solo formato, sin datos.

- **El aviso de contratos incompletos decía «falta capturar su importe»** cuando

lo que faltaba casi siempre era la vigencia. Ahora se deriva.

- **El enlace de firma vencido seguía entregando el contrato.** Cierra la lectura

al expirar, en las dos superficies públicas (`/api/firma/[token]` y la página `/firmar/[token]`).

- **Corregir la renta ya pone al día el calendario:** las cuotas no pagadas

toman el importe nuevo, que antes solo alcanzaba a los periodos nuevos.

- **El enlace de firma solo se entrega a quien tiene permiso de `crear`.** Con

`ver` se obtenía el token, y con el token se firma sin sesión.

- **ADR 0007 — los vencimientos se anclan al inicio del contrato.** `setMonth`

desbordaba en los meses cortos: un contrato del 29 se quedaba SIN cuota en febrero y desde ahí todo vencía el día 1. **Trae migración de datos** (ver el bloque 3B, y el aviso previo aquí abajo).

Verificado antes del deploy: build de producción en clon limpio ✓ · 393 pruebas en verde ✓ · migraciones probadas e idempotentes ✓ · migración 0007 ensayada contra una copia con `rollback` y luego aplicada a la base de demo, con segunda pasada en cero ✓

Antes de empezar — aviso al equipo de Arrendadores

Esto no es opcional y va ANTES del despliegue, no después.

La migración del ADR 0007 **mueve fechas de vencimiento ya generadas**. Un contrato del día 29 que hoy muestra sus pagos el día 1 pasará a mostrarlos el 29, y aparecerá un vencimiento en febrero que antes no existía. Quien haya exportado o enviado un calendario de pagos a un propietario verá que las fechas ya no coinciden con lo que envió.

Además, **el número de cuotas de un contrato puede cambiar**, y con él su renta total comprometida: la serie correcta no tiene por qué tener la misma longitud que la torcida. El P&L de una pantalla puede moverse.

Las dos cosas son el efecto buscado —el calendario pasa a decir lo que dice el contrato— pero si nadie avisa se leen como un error del sistema.

Propiedades que hacen seguro este deploy

- Las migraciones de ESQUEMA son aditivas (`add column if not exists` , `create table if not exists` , `add value if not exists`). El build viejo sigue funcionando con el esquema nuevo: las columnas que no conoce, no las consulta.
- La migración de DATOS (bloque 3B) no lo es, y cambia la historia del rollback respecto a la revisión 1 de este runbook: reescribe filas de `pagos_renta` . No modifica el esquema, así que el build viejo la tolera sin enterarse, pero **no se deshace sola**. El backup del bloque 1 deja de ser una precaución y pasa a ser el único camino de vuelta para esos datos.
- El punto de no retorno del código sigue siendo el bloque 6 (`pm2 reload`).

El de los datos es el bloque 3B.

- PM2 sirve el bundle que tenía al arrancar. El `git pull` ya hecho no afecta a nada hasta que se recargue.

Triggers de rollback

Concepto	Valor
Ventana de observación	30 min de vigilancia activa tras el reload
Rollback inmediato si	aparece <code>column ... does not exist</code> o <code>relation ... does not exist</code> en los logs
Rollback inmediato si	login o <code>/inicio</code> no devuelven 200 / 307 en el smoke test
Rollback inmediato si	más de 2% de 5xx en 5 minutos
Procedimiento (código)	checkout del commit previo, rebuild, <code>pm2 reload</code>
Procedimiento (datos)	<code>pg_restore</code> de <code>pagos_renta</code> desde el backup del bloque 1
Base de datos	se toca: el bloque 3B reescribe <code>pagos_renta</code>
Tiempo estimado	3 a 5 minutos el código; el restore depende del tamaño del dump

No revertir solo el código. Dejaría los calendarios realineados en la base y un binario que vuelve a torcerlos en cuanto alguien edite un contrato. Si hay que revertir, se revierten los dos o ninguno (ver «Cómo revertir» del ADR 0007).

Bloque 0 — Punto de rollback

Apunta el commit que sale aquí. Es a donde se vuelve si algo falla.

```
cd /var/www/Spaces
git reflog -5
git log --oneline -1
```

Commit previo anotado: _____

Bloque 1 — Backup

```
sudo -u postgres pg_dump -d spaces -Fc -f ~/spaces_$(date +%Y%m%d_%H%M).dump
ls -lh ~/spaces_*.dump | tail -1
```

Debe pesar del orden de MB. **Un `pg_dump` que falla deja un archivo de 0 bytes y la salida se ve casi igual**, por eso el `ls -lh`. Si falla, PARAR.

En esta revisión el backup es **load-bearing**: es la única forma de deshacer el bloque 3B. No sigas sin él.

Archivo de backup: _____

Bloque 2 — Qué falta en el esquema

Lectura: 1 = ya está aplicada · 0 = falta

```
sudo -u postgres psql -d spaces -c "
select 'contrato_firmas' o, count(*) ok from information_schema.tables where table_name='contrato_firmas'
union all select 'documento_congelado', count(*) from information_schema.columns where table_name='contratos_arrendamiento'
union all select 'tenants.rfc', count(*) from information_schema.columns where table_name='tenants' and column_name='rfc'
union all select 'ciudad_firma', count(*) from information_schema.columns where table_name='contratos_arrendamiento' and
union all select 'DIARIA', count(*) from pg_enum e join pg_type t on t.oid=e.enumtypid where t.typname='periodicidad_pago'
union all select 'licencias', count(*) from information_schema.tables where table_name='licencias'
order by 1;"
```

2B — Cuántos calendarios están torcidos (antes de tocar nada)

Esto no cambia nada: cuenta el alcance del bloque 3B para poder compararlo después. Anota el número.

```
sudo -u postgres psql -d spaces -c "
with contrato as (
  select c.id, c.fecha_inicio, c.fecha_fin,
         case c.periodicidad when 'MENSUAL' then 1 when 'BIMESTRAL' then 2
                             when 'TRIMESTRAL' then 3 when 'SEMESTRAL' then 6 when 'ANUAL' then 12 end as meses
  from contratos_arrendamiento c
  where c.fecha_fin is not null and c.monto_renta is not null
        and c.periodicidad in ('MENSUAL','BIMESTRAL','TRIMESTRAL','SEMESTRAL','ANUAL')),
correcto as (
  select c.id contrato_id,
         to_char((c.fecha_inicio + (k * c.meses || ' months')::interval)::date, 'YYYY-MM-DD') periodo
  from contrato c cross join lateral generate_series(0,
    ((extract(year from age(c.fecha_fin, c.fecha_inicio))::int*12
    + extract(month from age(c.fecha_fin, c.fecha_inicio))::int)/c.meses)+1) k
  where (c.fecha_inicio + (k*c.meses || ' months')::interval)::date <= c.fecha_fin)
select count(*) as calendarios_desalineados from (
  select contrato_id from (select contrato_id, periodo from correcto
                           except select contrato_id, periodo from pagos_renta) f
  union
```

```
select contrato_id from (select p.contrato_id,p.periodo from pagos_renta p
                        where p.contrato_id in (select contrato_id from correcto)
                        except select contrato_id,periodo from correcto) s) t;"
```

Calendarios desalineados antes: _____

Bloque 3 — Migraciones de esquema

Todas idempotentes: si alguna ya corrió, no hace nada. Se aplican con `psql` como

`postgres` y no con el runner de Node, para no depender de credenciales en

`.env`. `ON_ERROR_STOP` aborta al primer error.

```
sudo -u postgres psql -d spaces -v ON_ERROR_STOP=1 -f db/migrations/20260729_licencias_permisos.sql
sudo -u postgres psql -d spaces -v ON_ERROR_STOP=1 -f db/migrations/20260729_datos_contrato_documento.sql
sudo -u postgres psql -d spaces -v ON_ERROR_STOP=1 -f db/migrations/20260729_firma_contrato.sql
```

DIARIA va **sola y al final**: PostgreSQL no permite usar un valor de enum recién agregado dentro de la misma transacción que lo agrega.

```
sudo -u postgres psql -d spaces -v ON_ERROR_STOP=1 -f db/migrations/20260729_periodicidad_diaria.sql
```

Bloque 3B — Migración de DATOS (ADR 0007)

Punto de no retorno de los datos. Va después de las de esquema. Reescribe

`pagos_renta` ; no toca el esquema.

Guarda la salida en un archivo. El reporte de contratos que la migración NO pudo reparar solo existe ahí: no queda en ninguna tabla, y son los que necesitan que una persona los mire.

```
sudo -u postgres psql -d spaces -v ON_ERROR_STOP=1 \
-f db/migrations/20260731_calendario_meses_cortos.sql \
2>&1 | tee ~/migracion_0007_$(date +%Y%m%d_%H%M).log
```

Salida esperada:

```
NOTICE: Calendarios desalineados encontrados: N      ← el número del bloque 2B
NOTICE: Reparados: N
NOTICE: Sin tocar (requieren revision manual): 0
COMMIT
```

Qué hacer con cada línea:

- `Reparados` = `encontrados` y `Sin tocar`: 0 → todo bien, sigue.
- `Sin tocar` > 0 → NO es un fallo y no hay que parar. Son contratos con un

pago real en una fecha que el calendario correcto no contempla, o con una cuota impaga que lleva factura, comprobante u observaciones fuera de sitio. La migración prefiere dejarlos como están a borrar el comprobante que alguien subió. **Pasa la lista al**

equipo de Arrendadores para que decidan uno por uno.

- **ROLLBACK** o cualquier **ERROR** → la migración va en una transacción única,

así que no dejó nada a medias. PARAR y revisar.

Es idempotente: volver a correrla sobre una base ya migrada reporta 0 y no escribe nada.

Bloque 4 — Verificar

Repetir el bloque 2. **Todo debe salir en 1**. Si algo sigue en **0**, PARAR.

Repetir el bloque 2B. **Debe salir 0**, salvo por los contratos que el bloque 3B reportó como «sin tocar», que seguirán contándose porque efectivamente siguen desalineados a propósito.

Bloque 5 — Build

Todavía no afecta a lo que se está sirviendo.

```
npm ci
npm --prefix apps/web run build
```

Si muere por falta de memoria:

```
NODE_OPTIONS=--max-old-space-size=2048 npm --prefix apps/web run build
```

Es el bloque más frágil: `npm ci` borra `node_modules` y reinstala. Si se corta a la mitad, el build falla y ya no se puede recargar. Mientras tanto producción sigue sirviendo lo viejo sin enterarse.

Bloque 6 — Recarga (el cambio entra en vivo)

```
pm2 reload spaces-web
pm2 describe spaces-web | grep -iE 'status|uptime|restarts'
```

Bloque 7 — Smoke test

```
curl -s -o /dev/null -w "login: %{http_code}\n" http://localhost:3000/spaces-dooh/login/
curl -s -o /dev/null -w "inicio: %{http_code}\n" http://localhost:3000/spaces-dooh/inicio/
```

Esperado: `login` → **200** · `inicio` → **307** (redirige a login sin sesión). Ambos son correctos.

Después, en el navegador: entrar, abrir **Inventario** y **Arrendadores**, y abrir la ficha de una pantalla para confirmar que el margen se calcula con la renta.

Y lo propio de esta revisión, en **Arrendadores** → **Pagos de renta**:

- Las fechas de la columna «Vence» coinciden con el día del mes del contrato (no

van un día antes ni caen todas el día 1).

- Un contrato mensual del día 29, 30 o 31 tiene cuota en **todos** los meses de su vigencia, febrero incluido.
- El aviso de contratos incompletos nombra lo que falta de verdad (vigencia, periodicidad...) y no «su importe» si el importe está.

Bloque 8 — Vigilancia (30 min)

```
pm2 logs spaces-web --lines 100 | grep -iE "does not exist|error|ECONN"
```

Cualquier `column ... does not exist` o `relation ... does not exist` implica rollback inmediato.

Bloque 9 — Rollback (solo si hace falta)

Código:

```
git checkout <commit del bloque 0>
npm ci && npm --prefix apps/web run build && pm2 reload spaces-web
```

Datos (solo si además hay que deshacer el bloque 3B):

```
sudo -u postgres pg_restore -d spaces --data-only --table=pagos_renta \
--clean <backup del bloque 1>
```

Se pierde cualquier pago registrado entre el despliegue y la restauración: hay que recuperarlo a mano desde el log de Actividad.

Revertir el código sin revertir los datos deja el peor de los dos mundos: los calendarios quedan realineados y el binario viejo vuelve a torcerlos en cuanto alguien edite un contrato.

Pendientes conocidos

- **Deuda de datos:** tras el deploy, las pantallas sin contrato completo

aparecerán sin costo. Es el efecto buscado del ADR 0001, pero conviene avisar al equipo.

- **ADR 0006 Fase 2:** borrar las columnas `costo_compra` y el campo de `types.ts`; `seed.ts` y `mock.ts` aún inventan `costoCompra = tarifa * 0.62`.

- **Acortar la vigencia de un contrato** deja vivas las cuotas que quedan fuera

del nuevo rango: el generador inserta y reajusta importes, pero no borra. Es la misma familia de desincronización que arregló el ADR 0007 y pide la misma decisión (qué hacer si alguna ya está PAGADA).

- `pagos_renta.periodo` sigue siendo `text`. Con tipo `date` la aritmética sería nativa; es un cambio de esquema con backfill propio (ver ADR 0007).
- **Zona horaria del servidor:** si el droplet corre en UTC, las columnas `date`

llegan al navegador como `...T00:00:00.000Z` y un cliente en México las pinta un día antes — el mismo defecto que se arregló para `periodo`, por la otra punta. Comprobar con `timedatectl` y, si es UTC, abrirle su propia corrección (la de raíz es que el driver devuelva `date` como texto plano).

- Rotar la contraseña SSH de `emiliano` usada en la sesión del 2026-07-30.

Cerrado desde la revisión 1

- `~~ GET /api/firma/[token]` devuelve el documento sin comprobar `expirado` `~~ →`

cerrado en `99f3d59`, y de paso en la página `/firmar/[token]`, que era la otra superficie y no estaba en la lista.